Duco Core DR 脚本使用手册

1.2

中科新松有限公司

2026 年 06 月 08 日

Contents

1 简介 **1**

2 数据类型和变量 **2**
2.1 内置数据类型 2
2.2 系统常量 3
2.3 变量 4

3 表达式 **5**
3.1 算术运算表达式 5
3.2 关系运算表达式 5
3.3 逻辑运算表达式 5
3.4 赋值表达式 6
3.5 函数调用表达式 6

4 语句 **7**
4.1 循环语句 while 7
4.2 函数定义 7
4.3 return 语句 8
4.4 条件控制语句 8
4.5 goto 语句 9
4.6 break 语句 9

5 系统函数说明 **10**
5.1 移动控制 10
5.2 RS485 16
5.3 系统函数及外设 19
5.4 调试相关 31
5.5 数学运算函数 32
5.6 字符串相关函数 41
5.7 辅助函数 44

CHAPTER 1

简介

此脚本函数手册适用于新松协作机器人编程使用。

备注： 脚本函数名不能使用如下划线、标点符号等特殊字符作为起始字符。

CHAPTER 2

数据类型和变量

2.1 内置数据类型

空类型

nil，表示没有任何有效值

算术类型 number

系统内置算数类型为双精度实浮点数 (number)

字符串类型 string

例如"siasun robot"

布尔类型 boolean

例如 true、false

列表类型 list

可容纳其他数据类型的表 (数组结构)

例如 {1,2,3,4,5}、{"a","b","c"}

注：为了方便后续函数说明，定义以下类型

joints

特指长度为机器人关节自由度数量的描述各关节独立的相关数据信息（如关节位置）的双精度实浮点数列表，单位 rad，例如 {1,2,3,4,5,6}

pose

特指长度为 6 的描述笛卡尔空间位姿数据的双精度实浮点数列表，单位 m、rad，例如 {1,2,3,4,5,6}

num_list

特指双精度实浮点数列表，例如 {1.1,2.2,3,4}

pose_list

特指描述一组位姿数据的pose列表，例如{{1,2,3,4,5,6},{2,3,4,5,6,7}}

joints_list

特指描述一组关节相关数据的joints列表，例如{{1,2,3,4,5,6},{2,3,4,5,6,7}}

pose_speed

特指描述末端速度的双精度浮点数，单位 mm/s、m/s，例如 2.5

pose_acc

特指描述末端加速度的双精度浮点数，单位 mm/s^2、m/s^2，例如 2.5

joint_speed

特指描述关节最大运行速度的双精度浮点数，单位 deg/s、rad/s，例如 2.5

joint_acc

特指描述关节最大加速度的双精度浮点数，单位 deg/s^2、rad/s^2，例如 2.5

timer

特指描述时间间隔的双精度浮点数列表，单位 s，例如 1

2.2 系统常量

一个形如数字 42 的值被称作字面值常量。每个字面值常量都对应一种数据类型，字面值常量的形式和值决定了它的数据类型和在脚本中的使用方式。系统支持的常量如下。

基本类型常量

算数类型常量、true、false。

算数类型常量通常指的双精度实浮点数常量，该类型常量可以参与关系运算、算数运算、逻辑运算，并可以为基本类型变量进行赋值和初始化，亦可作为函数的实参进行参数传递。

true 在系统中代指真。

false 在系统中代指假。

字符串常量

字符串常量是字符的集合，字符串常量可用于字符串变量的定义，函数参数传递。

2.3 变量

变量在系统中通常用一个唯一的标识符来表示（id)。并且变量需要在定义之后才可以使用。

基本类型变量

```
a =1.23 --变量赋值上面定义了 a 的类型为双精度实浮点数，初始化 a 的值为 1.23
b = true -- b 布尔变量定义，初始化为真
c = false --c 布尔变量定义，初始化为假
```

字符串类型变量

```
s1="siasun" --s1 变量定义字符串类型初始化为"siasun"
s2="SiasunCobot" --s2 变量定义字符串类型初始化为"SiasunCobot"
s2=s1 --s2 变量赋值，值修改为"siasun"
```

列表类型变量

```
a1={} --a1 变量定义，空列表，长度为 0
a2={1.1,2.2,3.3,4.4} --a2 变量定义，双精度实浮点数列表，长度为 4
a3={"a","b","c"} --a3 变量定义，字符串列表，长度为 3
a4=a3 #a4 变量定义，a4 和 a3 指向同一个列表，修改 a4 的值，a3 同时被修改
```

可以通过索引操作具体的元素，索引下标从 1 开始，比如

```
a2[1]=12
a3[2]="siasun"
```

CHAPTER 3

表达式

表达式在系统中指的是有返回值的运算集合，是一个递归的定义。在此用 exp 表示表达式，用 op 表示运算符。

3.1 算术运算表达式

系统支持加（+）减（-）乘（*）除（/）四类算数运算，算数运算表达式返回算数类型常量。

3.2 关系运算表达式

系统支持 > < >= <= == ~= 类型的关系运算，关系运算表达式返回常量 True 或 False。

3.3 逻辑运算表达式

系统支持 and 、or 、not 类型的逻辑运算。

exp1 and exp2，若 exp1 为真而且 exp2 为真，那么表达式返回 true，否则返回 false。

exp1 or exp2，若 exp1 为真或者 exp2 为真，那么表达式返回 true，否则返回 false。

not exp1，若 exp1 为真，那么表达式返回 false，否则返回 true。

3.4 赋值表达式

变量名（id）= exp

赋值表达式的左值是一个变量的表示符，只有在变量已经定义过的情况下该表达式才是一个赋值表达式，若变量没有定义，那么就是一个变量定义语句。

3.5 函数调用表达式

函数名（实参列表）

CHAPTER 4

语句

4.1 循环语句 while

```
while (exp) do
exp
...
end
```

4.2 函数定义

```
function fun( 参数列表)
语句列表
...
return exp
end
```

例:

```
function fun1(a,b,c)
...
return a+b+c
end
```

参数类型：以实际传入参数为准

返回值类型：以实际返回参数为准

函数可以多参数返回，以”,”分割列如:

```
function fun1(a,b,c)
...
return a,b,c
end
```

4.3 return 语句

```
return exp
```

return 语句只能用在函数内部，用来返回函数的运算结果。

return 语句返回的数据类型决定了函数的返回值。若函数中没有包含 return 语句，那么函数不返回任何值。

4.4 条件控制语句

形式一：

```
if (exp) then
...
elseif (exp) then
...
else
...
end
```

形式二：

```
if (exp)
then
...
end
```

形式三：

```
if (exp) then
...
else
...
end
```

同大多数语言一样，系统支持 if 条件控制语句，当 if / elseif 中的判定条件表达式为真时，就执行语句块中的内容。

注意：控制结构的条件表达式结果可以是任何值，false 和 nil 为假，true 和非 nil 为真。特别注意，0 为 true：

```
if (0) then
print("0 is true")
end
>> 0 is true
```

4.5 goto 语句

```
goto Label
::Label:: exp
```

例：

```
a=1
::label:: print("----goto-----")
if a<3 then
goto label
end
>> ----goto-----
```

4.6 break 语句

使用 break 语句终止循环。

例：

```
a=10
while(a<20) do
print("a=",a)
a=a+1
if(a>15) then
break
end
end
```

CHAPTER 5

系统函数说明

5.1 移动控制

movej2(string: arm_name, joints: q, number: v, number: a, number: rad = 0, boolean: block = true)

函数说明:

该指令控制机械臂从当前状态，按照各关节相位同步运动的方式移动到目标关节角状态。

参数说明:

arm_name: string, 控制的机械臂的名称。

q : joints, 机械臂各关节的目标关节角度, 范围 [-2*PI, 2*PI], 单位 (rad)。

v : number, 最大关节角速度指令, 范围 [0.01*PI/180, 1.25*PI], 单位 (rad/s)。

a : number, 最大关节角加速度指令, 范围 [0.01*PI/180, ∞], 单位 (rad/s2)。

rad : number, 轨迹融合半径。预留。

block : boolean, 指令是否阻塞型指令, 如果为 false 表示非阻塞指令, 指令会立即返回, 默认为阻塞。

返回值:

无

示例:

```
movej2("arm_1", {-1.57,0.78,0,0.78,0,0,0},0.523,5.23,0)
```

movej_pose2(string: arm_name, pose: p, number: v, number: a, number: rad, joints: qnear, string: tool, string: wobj, boolean: block=true)

函数说明:

该指令控制机械臂从当前状态，按照各关节相位同步运动的方式移动到末端目标位置。

参数说明:

arm_name: string, 控制的机械臂的名称。

p : 目标机械臂工具在参考工件坐标系中的位姿，位置单位 m，姿态以 Rx、Ry、Rz 表示，范围 [-2*PI, 2*PI]，单位 (rad)。

v : 最大关节角速度，范围 [0.01*PI/180, 1.25*PI]，单位 (rad/s)。

a : 最大关节加速度，范围 [0.01*PI/180, ∞]，单位 (rad/s2)。

rad : 轨迹融合半径。预留。

qnear : 目标点附近位置对应的关节角度，用于确定逆运动学选解。

tool : 设置使用的工具的名称，默认为当前使用的工具。

wobj : 设置使用的工件坐标系的名称，默认为当前使用的工件坐标系。

block : 指令是否阻塞型指令，如果为 false 表示非阻塞指令，指令会立即返回，默认为阻塞。

返回值:

无

示例:

```
movej_pose2("arm_1", {0.52,-0.31,0.16,1.57,0,0},
0.5,5,0,{-1.57,0.78,0,0.78,0,0,0},"default","default")
```

---

movel(string: arm_name, pose: p, number: v, number: a, number: rad, joints: qnear, string: tool, string: wobj, boolean: block=true)

函数说明:

该指令控制机械臂末端从当前状态按照直线路径移动到目标位姿。

参数说明:

arm_name: string, 控制的机械臂的名称。

p : 目标机械臂工具在参考机械臂工件坐标系中的位姿，位置单位 m，姿态以 Rx、Ry、Rz 表示，范围 [-2*PI, 2*PI]，单位 (rad)。

v : 最大末端线速度，范围 [0.01, 5]，单位 (m/s)。

a : 最大末端线加速度，范围 [0.01, ∞]，单位 (m/s2)。

rad : 轨迹融合半径。预留。

qnear : 目标点附近位置对应的关节角度，用于校验机械臂运动过程中逆运动学选解空间。

tool ： 设置使用的工具的名称，默认为当前使用的工具。

wobj ： 设置使用的工件坐标系的名称，默认为当前使用的工件坐标系。

block ： 指令是否阻塞型指令，如果为 false 表示非阻塞指令，指令会立即返回，默认为阻塞。

返回值:

无

示例:

```
movel("arm_1", {0.45,-0.31,0.1,1.57,0,0},
0.5,5,0,{-1.57,0.78,0,0.78,0,0,0},"default","default")
```

---

movec(string: arm_name, pose: p1, pose: p2, number: v, number: a, number: rad, number: mode, joints: qnear, string: tool, string: wobj, boolean: block=true)

函数说明:

该指令控制机械臂做圆弧运动，起始点为当前位姿点，途径 p1 点，终点为 p2 点。

参数说明:

arm_name: string, 控制的机械臂的名称。

p1 ： 圆弧运动过程中任意机械臂工具在参考工件坐标系中的位姿中间点，位置单位 m，姿态以 Rx、Ry、Rz 表示范围 [-2*PI, 2*PI]，单位 (rad) 。

p2 ： 目标机械臂工具在参考工件坐标系中的位姿，位置单位 m，姿态以 Rx、Ry、Rz 表示范围 [-2*PI, 2*PI]，单位 (rad) 。

v ： 最大末端线速度，范围 [0.01, 5]，单位 (m/s)。

a ： 最大末端线加速度，范围 [0.01, ∞]，单位 (m/s2)。

rad ： 轨迹融合半径。预留。

mode ： 姿态控制模式。

0: 姿态与终点保持一致，即机械臂会以 p2 点的姿态为目标姿态，平滑运动到目标姿态。

1: 姿态与起点保持一致，即机械臂会以开始执行 movec 函数时机械臂末端工具坐标系在工件坐标系中的姿态为准，始终保持该姿态值。

2: 姿态受圆心约束，即机械臂会以开始执行 movec 函数时机械臂末端工具坐标系与目标圆弧路径起点处切线方向间关系为参考，在圆弧运动过程中始终保持末端工具与圆弧实时运动所处位置切线方向参考关系。

qnear ： 目标点附近位置对应的关节角度，用于校验机械臂运动过程中逆运动学选解空间。

tool ： 设置使用的工具的名称，默认为当前使用的工具。

wobj ： 设置使用的工件坐标系的名称，默认为当前使用的工件坐标系。

block ： 指令是否阻塞型指令，如果为 false 表示非阻塞指令，指令会立即返回，默认为阻塞。

返回值:

无

示例:

```
movec("arm_1", {0.45,-0.26,0.12,1.57,0,0},{0.45,-0.27,0.15,1.57,0,0},
0.1,1,0,0,{-1.17,0.33,0,1.81,0,-0.57,-0.39},"default","default",true)
```

---

move_circle (string: arm_name, pose: p1, pose: p2, number: v, number: a, number: rad, number: mode, joints: qnear, string: tool, string: wobj, boolean: block=true)

函数说明:

该指令控制机械臂做圆周运动，起始点为当前位姿点，途径 p1 点和 p2 点

参数说明:

arm_name: string, 控制的机械臂的名称。

p1 : 圆周运动过程中任意机械臂工具在参考机械臂工件坐标系中的位姿中间点 1，位置单位 m，姿态以 Rx、Ry、Rz 表示范围 [-2*PI, 2*PI]，单位 (rad) 。

p2 : 圆周运动过程中任意机械臂工具在参考机械臂工件坐标系中的位姿中间点 2，最终以机械臂初始运动位置-p1-p2 的顺序决定最终整圆轨迹，位置单位 m，姿态以 Rx、Ry、Rz 表示范围 [-2*PI, 2*PI]，单位 (rad) 。

v : 最大末端线速度，范围 [0.01, 5]，单位 (m/s)。

a : 最大末端线加速度，范围 [0.01, ∞]，单位 (m/s2)。

rad : 轨迹融合半径。预留。

mode : 姿态控制模式。

1: 姿态与起点保持一致，即机械臂会以开始执行 movec 函数时机械臂末端工具坐标系在工件坐标系中的姿态为准，始终保持该姿态值。

2: 姿态受圆心约束，即机械臂会以开始执行 movec 函数时机械臂末端工具坐标系与目标圆弧路径起点处切线方向间关系为参考，在圆弧运动过程中始终保持末端工具与圆弧实时运动所处位置切线方向参考关系。

qnear : 目标点附近位置对应的关节角度，用于校验机械臂运动过程中逆运动学选解空间。

tool : 设置使用的工具的名称，默认为当前使用的工具。

wobj : 设置使用的工件坐标系的名称，默认为当前使用的工件坐标系。

block : 指令是否阻塞型指令，如果为 false 表示非阻塞指令，指令会立即返回，默认为阻塞。

返回值:

无

示例:

```
move_circle("arm_1",{0.45,-0.26,0.12,1.57,0,0},{0.45,-0.27,0.15,1.57,0,0},
0.1,1,0,1,{-1.17,0.33,0,1.81,0,-0.57-0.39},"default","default",true)
```

---

tcp_move(string: arm_name, pose: pose_offset, number: v, number: a, number: rad, string: tool, boolean: block=true)

函数说明:

该指令控制机械臂沿工具坐标系直线移动一个增量。

参数说明:

arm_name : string, 控制的机械臂的名称。

pose_offset : pose 数据类型，或者长度为 6 的 number 型数组，表示工具坐标系下的位姿偏移量。偏移量将会转换为齐次变换矩阵右乘于当前机械臂末端位姿之上。

v : 最大末端线速度，范围 [0.01, 5]，单位 m/s，当 x、y、z 均为 0 时，线速度按比例换算成角速度。

a : 最大末端线加速度，范围 [0.01, ∞]，单位 (m/s2)。

rad : 轨迹融合半径。预留。

tool : 设置使用的工具的名称，默认为当前使用的工具。

block : 指令是否阻塞型指令，如果为 false 表示非阻塞指令，指令会立即返回，默认为阻塞。

返回值:

无

示例:

```
tcp_move("arm_1", {0,0,0.1,0,0,0},0.5,5,0 ,"default")
```

---

tcp_move_2p (string: arm_name, pose: p1, pose: p2 , number: v, number: a, number: rad, string: tool, string: wobj, boolean: block=true)

函数说明:

该指令控制机械臂沿工具坐标系直线移动一个增量，增量为 p1 与 p2 点之间的差，运动的目标点为：当前点 $*p1^{-1}*p2$。

参数说明:

arm_name : string, 控制的机械臂的名称。

p1 : 表示工具坐标系下的位姿偏移量计算点 1。

p2 : 表示工具坐标系下的位姿偏移量计算点 2。

v : 最大末端线速度，范围 [0.01, 5]，单位 (m/s)，当 x、y、z 均为 0 时，线速度按比例换算成角速度。

a : 最大末端线加速度，范围 [0.01, ∞]，单位 (m/s2)。

rad : 轨迹融合半径。预留。

tool : 设置使用的工具的名称，默认为当前使用的工具。

wobj ： 设置使用的工件坐标系的名称，默认为当前使用的工件坐标系。

block ： 指令是否阻塞型指令，如果为 false 表示非阻塞指令，指令会立即返回，默认为阻塞。

返回值:

无

示例:

```
tcp_move_2p("arm_1", {0.45,-0.27,0.15,1.57,0,0},{0.4,-0.3,0.12,1.57,0,0},
0.5,5,0,"default","default")
```

---

wobj_move(string: arm_name, pose: pose_offset, number: v, number: a, number: rad, string: wobj, boolean: block=true)

函数说明:

该指令控制机械臂沿工件坐标系直线移动一个增量。

参数说明:

arm_name ： string， 控制的机械臂的名称。

pose_offset ： pose 数据类型，或者长度为 6 的 number 型数组，表示工件坐标系下的位姿偏移量。

v ： 最大末端线速度，范围 [0.01, 5]，单位 m/s，当 x、y、z 均为 0 时，线速度按比例换算成角速度。

a ： 最大末端线加速度，范围 [0.01, ∞]，单位 (m/s2)。

rad ： 轨迹融合半径。预留。

wobj ： 设置使用的工件的名称，默认为当前使用的工件。

block ： 指令是否阻塞型指令，如果为 false 表示非阻塞指令，指令会立即返回，默认为阻塞。

返回值:

无

示例:

```
wobj_move("arm_1", {0,0,0.1,0,0,0},0.5,5,0 ,"default")
```

---

wobj_move_2p (string: arm_name, pose: p1, pose: p2 , number: v, number: a, number: rad, string: tool, string: wobj, boolean: block=true)

函数说明:

该指令控制机械臂沿工件坐标系直线移动一个增量，增量为 p1 与 p2 点之间的位姿偏移在工件坐标系下的描述 $offset_{wobj}$，运动的目标点为：当前点 * $offset_{wobj}$。

参数说明:

arm_name ： string， 控制的机械臂的名称。

p1 ： 表示工件坐标系下的位姿偏移量计算点 1。

p2 ： 表示工件坐标系下的位姿偏移量计算点 2。

v: 最大末端线速度，范围 [0.01, 5]，单位 (m/s)，当 x、y、z 均为 0 时，线速度按比例换算成角速度。

a ： 最大末端线加速度，范围 [0.01, ∞]，单位 (m/s2)。

rad ： 轨迹融合半径。预留。

tool ： 参考点使用的工具的名称，默认为当前使用的工具。

wobj ： 参考点使用的工件坐标系的名称，默认为当前使用的工件坐标系。

block ： 指令是否阻塞型指令，如果为 false 表示非阻塞指令，指令会立即返回，默认为阻塞。

返回值:

无

示例:

```
wobj_move_2p("arm_1", {0.45,-0.27,0.15,1.57,0,0},{0.4,-0.3,0.12,1.57,0,0},
0.5,5,0,"default","default")
```

5.2 RS485

tool_read_raw_data_485 (string: arm_name, number: len, number: timeout=3000)

函数说明:

机械臂末端 485 端口读取长度为 len 的字节数据，若超时，返回当前接收到的数据

参数说明:

arm_name: string，机械臂的名称。

len: number，需要读取的长度。

timeout: number，超时时间，单位 ms，默认 3000ms

返回值:

number_list，读取到的数据，未读到数据返回空列表。

示例:

```
list= tool_read_raw_data_485 ("arm_1", 10)
```

---

tool_read_raw_data_485 (string: arm_name, number_list: head, number_list: tail, number: timeout=3000)

函数说明:

机械臂末端 485 匹配头 head 和尾 tail 读取到一帧匹配的数据。若超时，返回当前接收到的数据

参数说明:

arm_name: string，机械臂的名称。

head: 需要匹配的头数据。

tail: 需要匹配的尾数据。

timeout: number，超时时间，单位 ms，默认 3000ms

返回值:

number_list 读取到的数据，未读到数据返回空列表。

示例:

```
list= tool_read_raw_data_48 ("arm_1", {255,1}, {1,255})
```

---

tool_read_raw_data_485 (string: arm_name, number_list: head, number: len, number: timeout=3000)

函数说明:

机械臂末端 485 匹配头 head 后读取到长度为 len 的一帧数据。若超时，返回当前接收到的数据

参数说明:

arm_name: string，机械臂的名称。

head: 需要匹配的头数据。

len: 需要读取的长度。

timeout: number，超时时间，单位 ms，默认 3000ms

返回值:

number_list 读取到的数据，未读到数据返回空列表。

示例:

```
list= tool_read_raw_data_485 ("arm_1", {255,1}, 10)
```

---

tool_write_raw_data_485 (string: arm_name, number_list: value)

函数说明:

末端 485 写原生数据，将表 value 中的数据写入 485 端口。

参数说明:

arm_name: string，机械臂的名称。

value: 需要写入的数据列表。

返回值:

true: 成功。

false: 失败。

示例:

```
tool_write_raw_data_485 ("arm_1", {1,2,3})
```

---

tool_write_raw_data_485 (string: arm_name, number_list: value, number_list: head)

函数说明:

末端 485 写原生数据，将表 value 中的数据加上 head 写入 485 端口。

参数说明:

arm_name: string，机械臂的名称。

value: 需要写入的数据列表。

head: 需要添加的头。

返回值:

true: 成功。

false: 失败

示例:

```
tool_write_raw_data_485 ("arm_1", {1,2,3}, {255,255})
```

---

tool_write_raw_data_485 (string: arm_name, number_list: value, number_list: head, number_list: tail)

函数说明:

末端 485 写原生数据，将表 value 中的数据加上头 head 和尾 tail 写入 485 端口。

参数说明:

arm_name: string，机械臂的名称。

value: 需要写入的数据列表。

head: 需要添加的头。

tail: 需要添加的尾

返回值:

true: 成功。

false: 失败。

示例:

```
tool_write_raw_data_485 ("arm_1", {1,2,3}, {255,255},
{255,255})
```

5.3 系统函数及外设

！ 注意

若范围参数越界，函数将返回 false 或 0

sleep(number: time)

函数说明:

该函数会让程序暂停一定时间。

参数说明:

time ： 需要暂停的时间，单位 (ms)。

返回值:

无

示例:

```
sleep (1000)
```

---

set_tool_digital_out (string: arm_name, number: portnum, boolean: val)

函数说明:

该函数可控制机械臂末端的 IO 输出口的高低电平。

参数说明:

arm_name: string，机械臂的名称。

portnum ： 机械臂末端的 IO 输出口序号，范围从 1-2。

val ： true 为高电平，false 为低电平。

参数错误时函数不改变 IO 输出。

返回值:

无

示例:

```
set_tool_digital_out ("arm_1", 1, true)
```

---

get_tool_digital_in (string: arm_name, number: portnum)

函数说明:

该函数可读取机械臂末端的 IO 输入口的高低电平，返回 true 为高电平，false 为低电平。

参数说明:

arm_name: string，机械臂的名称。

portnum: 机械臂末端的 IO 输入口序号，范围从 1-2。

返回值:

boolean，返回 true 为高电平，false 为低电平。

示例：

```
value = get_tool_digital_in ("arm_1", 1)
```

---

get_tool_digital_out (string: arm_name, number: portnum)

函数说明：

该函数可读取机械臂末端的 IO 输出口的高低电平，返回 true 为高电平，false 为低电平。

参数说明：

arm_name：string，机械臂的名称。

portnum：机械臂末端的 IO 输出口序号，范围从 1-2。

返回值：

boolean，返回 true 为高电平，false 为低电平。

示例：

```
value = get_tool_digital_out ("arm_1", 1)
```

---

set_wobj_offset (string: arm_name, boolean: val, number_list: wobj_offset)

函数说明：

基于当前的工件坐标系设置一个偏移量，后续该机械臂的 move 类脚本的参考工件坐标系上都将添加这个偏移量。该偏移量以右乘的形式叠加在当前工件坐标系的初始值上。

参数说明：

arm_name：string，机械臂的名称。

val：true 为启用偏移量，false 为取消偏移量。

wobj_offset：{x, y, z, rx, ry, rz} 工件坐标系偏移量（单位：m，rad）。

返回值：

无

示例：

```
ret= set_wobj_offset ("arm_1", true, {0.1,0,0,0,0,0})
```

---

set_tool_data(string: arm_name, string: name, number_list: tool_offset, number_list: load)

函数说明：

该函数会设置当前生效工具信息，设置成功后，若后续运动函数中的 TCP 设置为空（即使用当前坐标系）时，使用该工具参数。

参数说明：

arm_name: string，机械臂的名称。

name: 工具的名字，类型 string，最长不超过 32 个字节长度。

tool_offset: 工具 TCP 偏移量 {x_off, y_off,z_off,rx,ry,rz}，单位 (m, rad)。

load: 末端工具质量，质心，{mass,x_cog,y_cog,z_cog}，相对于法兰坐标系，单位 (kg, m)。

返回值:

无

示例:

```
ret= set_tool_data ("arm_1", "default", {0.1,0,0,0,0,0},
{5,0,0.05,0})
```

警告: 该函数只会临时修改当前控制器软件中生效的工具的信息。即使目标工具名称与当前机器人系统中已存在工具名称相同，也不会修改该工具的原始参数信息。当其他函数以名称的方式在此访问并使用该工具时，其相关信息仍为原始信息。若想要永久性修改目标工具信息，请使用 set_tool_data_workcell 代替该函数。

---

set_wobj(string: name, number_list: wobj_offset)

函数说明:

该函数会设置当前生效工件坐标系信息，设置成功后，若后续运动函数中的 wobj 设置为空（即使用当前坐标系）时，使用该工件坐标系参数。

参数说明:

name: 工件坐标系的名字，类型 string，最长不超过 32 个字节长度。

wobj_offset: 工件坐标系偏移量 {x_off,y_off,z_off,rx,ry,rz}，单位 (m, rad)。

返回值:

无

示例:

```
ret= set_wobj ("default", {0.1,0,0,0,0,0})
```

警告: 该函数只会临时修改当前控制器软件中生效的工件坐标系信息。即使目标工件坐标系名称与当前机器人系统中已存在工件坐标系名称相同，也不会修改该工件坐标系的原始参数信息。当其他函数以名称的方式在此访问并使用该工件坐标系时，其相关信息仍为原始信息。若想要永久性修改目标工件坐标系信息，请使用 set_wobj_workcell 代替该函数。

---

set_load_data (string: arm_name, number_list: load)

函数说明:

设置抓取负载。可以在程序运行过程中设置机械臂当前除工具以外的负载信息（质量、质心）。

参数说明：

arm_name：string，机械臂的名称。

load：末端工具抓取负载质量，质心，{mass,x_cog,y_cog,z_cog}，相对于工具坐标系，质量范围 [0, 5]，单位（kg，m）。

返回值：

无

示例：

```
ret= set_load_data ("arm_1", {5,0,0.05,0})
```

警告： 该函数会永久性改变当前除工具外的负载信息，即程序结束运行后仍然会保持。因此若出现运行该函数且程序停止运行后力控相关功能异常的情况，优先确认是否负载信息未被正确复位。

---

cal_ikine(string: arm_name, pose: p, joints: q_near, number_list: tcp, number_list: wobj)

函数说明：

基于目标工具在工件坐标系中的笛卡尔空间位姿，通过机器人逆运动学计算机械臂对应的关节角位置，在求解过程中，会选取靠近参考关节角位置或当前机械臂关节位置的解。

参数说明：

arm_name：string，机械臂的名称。

p：需要计算的末端位姿在设置工件坐标系的值，包含当前有效的工具偏移量，位置单位 m，姿态单位 rad。

q_near：用于计算逆运动学的参考关节位置，为空时使用当前关节值。

tcp：工具坐标系信息，tcp 偏移量 {x_off,y_off,z_off,rx,ry,rz}，(单位：m，rad)，为空使用当前工具。

wobj：工件坐标系相对于世界坐标系的位移 {x，y，z，rx，ry，rz}，(单位：m，rad)，为空使用当前工件坐标系。

返回值：

joints，返回的关节位置列表 {q1,q2,q3,q4,q5,q6,q7}

示例：

```
ret= cal_ikine("arm_1", {0.4,-0.5,-0.13,-1.57,0.3,0},{},{},{})
```

---

cal_fkine(string: arm_name, joints: axis, number_list: tcp, number_list: wobj)

函数说明：

基于目标机器人关节角位置，通过机器人正运动学计算目标工具在目标工件坐标系中的笛卡尔空间位姿。

参数说明:

arm_name: string，机械臂的名称。

axis: 目标机器人关节角位置，单位 (rad)。

tcp: 目标工具坐标系信息，TCP 偏移量 {x_off,y_off,z_off,rx,ry,rz}，(单位: m，rad)，为空使用当前工具。

wobj: 目标工件坐标系相对于世界坐标系的偏移量 {x, y, z, rx, ry, rz}，(单位: m，rad)，为空使用当前工件。

返回值:

pose，返回末端姿态列表 {x,y,z,rx,ry,rz}

示例:

```
ret= cal_fkine("arm_1", {1,0.2,0.4,0.8,-0.1,0.6,0.13},{},{})
```

---

get_tcp_pose(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂当前生效工具在基坐标系下的位姿。

参数说明:

arm_name: string，机械臂的名称。

返回值:

pose，返回末端位置 pose。

示例:

```
ret= get_tcp_pose("arm_1")
```

---

get_tcp_pose_coord(string: arm_name, string: tool, string: wobj)

函数说明:

该函数可获取末端法兰在目标工具坐标系参考目标工件坐标系下的位姿。

参数说明:

arm_name: string，机械臂的名称。

tool: 工具坐标系名称，默认为当前使用的坐标系。

wobj: 工件坐标系名称，默认为当前使用的坐标系。

返回值:

number_list，末端法兰的位姿，单位 (m, rad)。

示例:

```
ret= get_tcp_pose_coord ("arm_1", "tool_1", "wobj_1")
```

---

get_tcp_speed(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂当前生效工具末端原点的空间速度。

参数说明:

arm_name: string，机械臂的名称。

返回值:

number_list，末端速度列表，单位 m/s,rad/s。

示例:

```
ret= get_tcp_speed("arm_1")
```

---

get_tcp_acceleration(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂工具当前生效工具末端原点的空间加速度。

参数说明:

arm_name: string，机械臂的名称。

返回值:

number_list，末端加速度列表，单位 (m/s2, rad/s2)。

示例:

```
ret= get_tcp_acceleration("arm_1")
```

---

get_tcp_force(string: arm_name)

函数说明:

该函数可获取当前机械臂工具当前生效工具末端原点的空间力信息。该函数当且仅当安装了末端力传感器并正确配置启用时有效。

参数说明:

arm_name: string，机械臂的名称。

返回值:

number_list，返回末端力矩信息，{Fx,Fy,Fz,Mx,My,Mz}，单位 N、N.m。

示例:

```
ret= get_tcp_force("arm_1")
```

---

get_tcp_force_tool(string: arm_name, string: tool)

函数说明:

该函数可获取机械臂工具末端在目标工具坐标系下的力矩信息。该函数当且仅当安装了末端力传感器并正确配置启用时有效。

参数说明:

arm_name: string，机械臂的名称。

tool: 工具坐标系名称，默认为当前使用的坐标系。

返回值:

number_list, 返回工具坐标系下的力矩信息，{Fx,Fy,Fz,Mx,My,Mz}, 单位(N、N.m)。

示例:

```
ret= get_tcp_force_tool ("arm_1", "tool_1")
```

---

get_tcp_offset(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂有效的末端工具的偏移量。

参数说明:

arm_name: string，机械臂的名称。

返回值:

number_list,{x_off,y_off,z_off,rx,ry,rz} 返回 TCP 偏移量信息,单位 (m, rad)。

示例:

```
ret= get_tcp_offset("arm_1")
```

---

get_tool_load(string: arm_name, string: name)

函数说明:

该函数可获取当前设置工具的有效负载质量、质心位置和惯性张量。

参数说明:

arm_name: string，机械臂的名称。

name: string，工具的名称，可缺省，缺省时为获取当前使用的工具的质量、质心位置

返回值:

number_list, 质量单位 kg, 质心位置单位 (m), {mass,x_cog,y_cog,z_cog}。

示例:

```
ret= get_tcp_load("arm_1")
```

---

get_wobj(string: arm_name)

函数说明:

该函数可获取机械臂当前使用的工件坐标系的值

参数说明:

arm_name: string，机械臂的名称。

返回值:

number_list, {x, y, z, rx, ry, rz} 工件坐标系相对于世界坐标系的偏移量，单位 (m, rad)。

示例:

```
ret= get_wobj("arm_1")
```

---

get_actual_joints_position(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂各关节位置角度。

参数说明:

arm_name: string，机械臂的名称。

返回值:

返回 1-7 轴关节角度列表，单位 (rad)。

示例:

```
ret= get_actual_joints_position("arm_1")
```

---

get_target_joints_position(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂各关节的规划角度。

参数说明:

arm_name: string，机械臂的名称。

返回值:

返回 1-7 轴关节角度列表，单位 (rad)。

示例:

```
ret= get_target_joints_position("arm_1")
```

---

get_actual_joints_speed(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂各关节角速度。

参数说明:

arm_name: string，机械臂的名称。

返回值:

返回 1-7 轴关节速度列表，单位 (rad/s)。

示例:

```
ret= get_actual_joints_speed("arm_1")
```

---

get_target_joints_speed(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂各关节的规划角速度。

参数说明:

arm_name: string，机械臂的名称。

返回值:

返回 1-7 轴关节速度列表，单位 (rad/s)。

示例:

```
ret= get_target_joints_speed("arm_1")
```

---

get_actual_joints_acceleration(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂各关节角加速度。

参数说明:

arm_name: string，机械臂的名称。

返回值:

返回 1-7 轴关节加速度列表，单位 (rad/s2)。

示例:

```
ret= get_actual_joints_acceleration("arm_1")
```

---

get_target_joints_acceleration(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂各关节的规划角加速度。

参数说明:

arm_name: string，机械臂的名称。

返回值:

返回 1-7 轴关节加速度列表，单位 (rad/s2)。

示例:

```
ret= get_target_joints_acceleration("arm_1")
```

---

get_actual_joints_torque(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂各关节力矩实际值。

参数说明:

arm_name: string，机械臂的名称。

返回值:

返回 1-7 轴关节力矩列表，单位 (N.m)。

示例:

```
ret= get_actual_joints_torque("arm_1")
```

---

get_target_joints_torque(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂各关节目标力矩。

参数说明:

arm_name: string，机械臂的名称。

返回值:

返回 1-7 轴关节力矩列表，单位 (N.m)。

示例:

```
ret= get_target_joints_torque("arm_1")
```

---

get_flange_pose(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂末端法兰在基坐标系下的位姿。

参数说明:

arm_name: string，机械臂的名称。

返回值:

末端法兰位置 pose。

示例:

```
ret= get_flange_pose("arm_1")
```

---

get_flange_speed(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂末端法兰在基坐标系下的速度。

参数说明:

arm_name: string，机械臂的名称。

返回值:

末端法兰速度列表，单位 (m/s, rad/s)。

示例:

```
ret= get_flange_speed("arm_1")
```

---

get_flange_acceleration(string: arm_name)

函数说明:

该函数可获取当前状态下机械臂末端法兰在基坐标系下的加速度。

参数说明:

arm_name: string，机械臂的名称。

返回值:

末端法兰加速度列表，单位 (m/s2, rad/s2)。

示例:

```
ret= get_flange_acceleration("arm_1")
```

---

sub_program(string : program_name, string: tree_name)

函数说明:

调用子程序。

参数说明:

program_name: 子程序名。

tree_name: 子程序树节点名称，显示使用。

返回值:

无

示例:

```
sub_program("program1.jspf", " ")
```

---

collision_detect (number: level)

函数说明:

设置碰撞检测等级。

参数说明:

level：0：关闭碰撞检测，1-5：对应设置碰撞检测等级 1 到等级 5。

返回值：

无

示例：

```
collision_detect (1)
```

---

set_value (string: name, var: data)

函数说明：

设置系统变量值。

参数说明：

name：系统变量名称

var：设置的系统变量值，根据对应的系统变量类型确定其类型

当系统变量类型为：

boolean,data 类型为 boolean;

number, data 类型为 number;

string, data 类型为 string;

number_list, data 类型为 number_list;

pose, data 类型为 number_list;

joints, data 类型为 number_list;

返回值：

无

示例：

```
set_value ("g_data",true)
```

---

get_value (string: name)

函数说明：

获取系统变量值。

参数说明：

name：系统变量名称

返回值：

ret：根据对应的系统变量类型确定其返回类型，当系统变量类型为：

boolean, ret 类型为 boolean;

number, ret 类型为 number;

string, ret 类型为 string;

number_list, ret 类型为 number_list;

pose, ret 类型为 number_list;

joints, ret 类型为 number_list;

示例:

```
ret = get_value ("g_data")
```

---

get_robot_state(string: arm_name)

函数说明:

获取机械臂状态信息。

参数说明:

arm_name: string, 机械臂的名称。

返回值:

number_list: 机械臂状态信息列表, number_list[1] 表示机械臂状态, number_list[2] 表示运行状态

示例:

```
get_robot_state("arm_1")
```

5.4 调试相关

log(string: describe, val)

函数说明:

该函数可在程序中插入 log 日志, 记录程序运行中的相关信息。

参数说明:

describe : 需要在日志中记录的描述信息。

val : 需要在 describe 信息后记录的变量值信息, 会自动转换为 string 类型并被记录, 可以是 number,string,list 等任意类型。

返回值:

无

示例:

```
log("this is log",{1,2,3,4})
```

---

message(string: describe)

函数说明:

该函数可在程序运行过程中产生弹窗, 显示目标描述信息, 并暂停当前程序。

参数说明:

describe : 弹窗所显示的目标描述信息。

返回值:

无

示例:

```
message ("this is message")
```

5.5 数学运算函数

abs(number: number)

函数说明:

计算绝对值。

参数说明:

number: number 数据类型。

返回值:

number 类型。

示例:

```
ret = abs (2)
```

---

cos(number: number)

函数说明:

计算余弦值。

参数说明:

number: number 数据类型，单位 (rad)。

返回值:

number 类型。

示例:

```
ret = cos (1.57)
```

---

acos(number: number)

函数说明:

计算反余弦值。

参数说明:

number: number 数据类型，余弦值，取值范围 [-1,1]。

返回值:

number 类型，单位 (rad)。

示例:

```
ret = acos (0.5)
```

---

sin(number: number)

函数说明:

计算正弦值。

参数说明:

number: number 数据类型，单位 (rad)。

返回值:

number 类型。

示例:

```
ret = sin (1.57)
```

---

asin(number: number)

函数说明:

计算反正弦值。

参数说明:

number: number 数据类型，正弦值，取值范围 [-1,1]。

返回值:

number 类型，单位 (rad)。

示例:

```
ret = asin (0.5)
```

---

tan(number: number)

函数说明:

计算正切值。

参数说明:

number: number 数据类型，单位 (rad)。

返回值:

number 类型。

示例:

```
ret = tan (1.57)
```

---

atan(number: number)

函数说明:

计算反正切值。

参数说明:

number: number 数据类型，正切值。

返回值:

number 类型，范围 (-pi/2,+pi/2)，单位 (rad)。

示例:

```
ret = atan (0.5)
```

---

atan2(number: number1, number: number2)

函数说明:

计算 number1/number2 的反正切值，根据 number1、number2 的符号，确定返回值所在的象限。

参数说明:

number1: number 数据类型。

number2: number 数据类型。

返回值:

number 类型，范围 (-pi,pi)，单位 (rad)。

示例:

```
ret = atan2 (1,2)
```

---

pow(number: base, number: exponent)

函数说明:

幂运算。

参数说明:

base: 底数。

exponent: 指数。

注: 如果 base 为负值并且 exponet 不是一个整数值，则发生定义域错误。当 base 为 0 且 exponent 小于 0 时，会发生定义域错误。

返回值:

number 类型。

示例:

```
ret = pow (10,2)
```

---

sqrt(number: number)

函数说明:

计算平方根。

参数说明:

number: number 数据类型，取值大于等于 0。

返回值:

number 类型。

示例:

```
ret = sqrt (10)
```

---

deg2rad(number: number)

函数说明:

角度值转换成弧度值。

参数说明:

number: number 数据类型，单位度。

返回值:

number 类型，单位 (rad)。

示例:

```
ret = deg2rad (90)
```

---

rad2deg(number: number)

函数说明:

弧度值转换成角度值。

参数说明:

number: number 数据类型，单位 (rad)。

返回值:

number 类型，单位度。

示例:

```
ret = rad2deg (1.57)
```

---

pose_trans (pose: p1, pose: p2)

函数说明:

坐标变换函数，位姿 p1，经过 p2 变换得到一个新的位姿，返回的位姿 ret=p1*p2。

参数说明:

p1: pose 类型，姿态信息按照 Rz*Ry*Rx 方式计算旋转矩阵，位置单位 m，姿态单位 (rad)。

p2: pose 类型，姿态信息按照 Rz*Ry*Rx 方式计算旋转矩阵，位置单位 m，姿态单位 (rad)。

返回值:

pose 类型，经过坐标变换后的位姿，位置单位 m，姿态单位 (rad)。

示例：

```
ret = pose_trans ({492,140.5,442,-3.14,0,-1.57},
{231.4,140.5,582.3,-3.14, 0,-1.57} )
```

---

pose_inv (pose: p1)

函数说明：

坐标逆变换函数，求解一个变换的逆变换。

参数说明：

p1：pose 类型，姿态信息按照 Rz*Ry*Rx 方式计算旋转矩阵，位置单位 m，姿态单位 (rad)。

返回值：

pose 类型，逆变换的解。姿态信息按照 Rz*Ry*Rx 方式计算旋转矩阵，位置单位 m，姿态单位 (rad)。

示例：

```
ret = pose_inv ({492,140.5,442,-3.14,0,-1.57} )
```

---

pose_distance(pose: p1, pose: p2)

函数说明：

计算两点之间的空间距离，姿态不参与计算。

参数说明：

p1：要计算的点 1。

p2：要计算的点 2。

返回值：

number：两点间的距离。

示例：

```
ret = pose_distance ({492,140.5,442,-3.14,0,-1.57},
{231.4,140.5,582.3, -3.14,0,-1.57} )
```

---

pose_offset(pose: p1, pose: p2)

函数说明：

计算两个位姿之间的偏移量，计算方式为 ret=$p1^{-1}$*p2。

参数说明：

p1：要计算的点 1。

p2：要计算的点 2。

返回值：

两个位姿之间的偏移量。

示例：

```
ret = pose_offset ({492,140.5,442,-3.14,0,-1.57},
{231.4,140.5,582.3, -3.14,0,-1.57} )
```

---

num2str (number: num, number: precision)

函数说明：

将数值按照指定的精度转换为字符串。比如 num2str(1.23456, 3) 转换成“1.235”，num2str(1.23) 转换成“1.230000”。

参数说明：

num：要转换的数值。

precision：保留的精度，可缺省，默认为 6。

返回值：

string

示例：

```
ret = num2str (1.234, 2)
```

---

str2num (string: s)

函数说明：

将字符串转换成数值，比如“1.23”转换成 1.23。

参数说明：

s：需要转译的字符串。

返回值：

number 类型，转译后的数。

示例：

```
ret = str2num (“1.234”)
```

---

list2str (number_list: a)

函数说明：

将 number 类型列表转换为固定格式字符串，字符串以“(”开头，“)”结尾，中间的数据之间使用“,”分隔。比如：列表 {1.23,1.57,2.3} 会转换成字符串“(1.23,1.57,2.3)”。

参数说明：

a ：number 类型的列表。

返回值：

string

示例:

```
ret = list2str ({1.2,3.4,5.6})
```

---

str2list(string: s)

函数说明:

将字符串转换为列表，字符串以“(”开头，“)”结尾，中间的数据之间使用“,”分隔。比如：字符串“(1.23,1.57,2.3)”会转换成列表 {1.23,1.57,2.3}。

参数说明:

s ： 需要转换的格式化字符串。

返回值:

number_list，如果转换错误，则返回空列表。

示例:

```
ret = str2list (“(1.2,3.4,5.6)”)
```

---

get_pose_trans(pose: p)

函数说明:

获取机器人位姿中关于位置的偏移量。

参数说明:

p ： pose 类型，机器人位姿数据，单位：m、rad。

返回值:

number_list，返回 pose 的 3 维位置偏移量信息，单位：m。

示例:

```
ret = get_pose_trans ({1,2,3,4,5,6})
```

期望结果:ret={1,2,3}

---

get_pose_rpy(pose: p)

函数说明:

获取机器人位姿中关于姿态的偏移量。

参数说明:

p: pose 类型，机器人位姿数据，单位：m、rad。

返回值:

number_list，返回 pose 的 3 维姿态偏移量信息，单位：rad。

示例:

```
ret = get_pose_rpy({1,2,3,4,5,6})
```

期望结果:ret={4,5,6}

---

get_number_list_norm(number_list: nl)

函数说明:

获取 nl 中所有数据的模长，方式为取 nl 所有元素的平方和。

参数说明:

nl : number_list 类型，输入参数。

返回值:

number，输入参数的模长。

示例:

```
ret = get_number_list_norm ({1,2,3,4,5,6})
```

---

normalize_number_list(number_list: nl)

函数说明:

将输入的 nl 归一化并返回输出，方式为 nl 的所有元素除以其模长。

参数说明:

nl : number_list 类型，输入参数。

返回值:

number_list，归一化后的数据。

示例:

```
ret = normalize_number_list ({1,2,3,4,5,6})
```

---

number_list_cross(number_list: nl1, number_list: nl2)

函数说明:

计算两个维度为 3 的 number_list 的叉乘，计算方式为 nl1×nl2。

参数说明: nl1 : number_list 类型，输入参数，维度为 3。

nl2 : number_list 类型，输入参数，维度为 3。

返回值:

number_list，叉乘结果。

示例:

```
ret = number_list_cross ({1,2,3},{4,5,6})
```

---

number_list_cross(list: nl)

函数说明:

计算的 nl 中所有 number_list 的叉乘。

参数说明:

nl : list 类型，{number_list: nl1, number_list: nl2, ...}，所有 number_list 的维度必须大于等于 3 且相等，且 number_list 的参数个数必须为 number_list 的维度-1。

返回值:

number_list，叉乘结果，维度与 nl 的元素的维度相同。

示例:

```
ret = number_list_cross({{1,2,3},{4,5,6})
```

---

number_list_dot(number_list: nl1, number_list: nl2)

函数说明:

计算 nl1 和 nl2 的点积。

参数说明:

nl1: number_list 类型，输入参数。

nl2: number_list 类型，维度与 nl1 相同。

返回值:

number，nl1 · nl2 的值。

示例:

```
ret = number_list_dot ({1,2,3,4,5,6},{1,2,3,4,5,6})
```

---

rpy_to_axial_angle(number_list: rpy)

函数说明:

计算输入 rpy 所对应的轴角，rpy 默认为动态 ZYX。

参数说明:

rpy : number_list 类型，默认参考动态 ZYX，单位 rad，必须为 3 维。

返回值:

number_list，rpy 所对应的轴角，维度为 4 维，第 1 个值对应参考旋转轴矢量的旋转角度，单位 rad，后三个值为所对应的旋转轴矢量。

示例:

```
ret = rpy_to_axial_angle({1,2,3})
```

---

rpy_to_rot(number_list: rpy)

函数说明:

计算输入 rpy 所对应的旋转矩阵，rpy 默认为动态 ZYX。

参数说明:

rpy : number_list 类型，默认参考动态 ZYX，单位 rad，必须为 3 维。

返回值:

list，{number_list，number_list，number_list}，按 RX、RY、RZ 的顺序返回 rpy 所对应的 3 组正交基，组成对应的旋转矩阵。

示例:

```
ret = rpy_to_rot ({1,2,3})
```

---

rot_to_rpy(list: rot)

函数说明:

计算输入旋转矩阵 rot 所对应 rpy，rpy 默认为动态 ZYX。

参数说明:

rpy : list 类型,{number_list，number_list，number_list},按 RX、RY、RZ 的顺序输入三组正交基，numberl_list 必须为 3 维。

返回值:

number_list，RX、RY、RZ 值。

示例:

```
ret = rot_ro_rpy ({{1,2,3},{4,5,6},{7,8,9}})
```

5.6 字符串相关函数

str_cat(string : str1, string : str2)

函数说明:

拼接两个字符串。

参数说明:

str1 : 要拼接的左侧字符串。

str2 : 要拼接的右侧字符串。

返回值:

拼接后的字符串。

示例:

```
ret = str_cat ("a", "b")
```

---

str_cmp(string : str1, string : str2)

函数说明:

比较两个字符串是否相等，系统支持的字符串最大长度为 255。

参数说明:

str1 : 要比较的字符串 1。

str2 : 要比较的字符串 2。

返回值:

false: 不相等。

true: 相等。

示例:

```
ret = str_cmp ("a", "b")
```

---

str_del(string : str, number: index, number: num)

函数说明:

删除字符串中的一段。

参数说明:

str : 要进行操作的字符串。

index: 表示从第几位开始删除，从 0 开始计数。

num : 表示删除几位。

返回值:

删除后的字符串。

示例:

```
ret = str_del ("abcde", 2, 2)
```

---

str_insert(string : str1, number: index, number: num, string : str2)

函数说明:

在一个字符串中插入另一个字符串。

参数说明:

str1 : 进行操作的基准字符串。

index : 从基准字符串哪一位开始插入，从 0 开始计数。

num : 插入多少位。

str2 : 插入的字符串。

返回值:

操作后的字符串。

示例:

```
ret = str_insert ("abcde", 2, "fff")
```

---

str_substr(string: str, number: index, number: num)

函数说明:

取字符串的子集。

参数说明:

str : 要操作的字符串。

index : 从哪一位开始取，从 0 开始计数。

num : 取多少位。

返回值:

操作后的字符串。

示例:

```
ret = str_substr ("abcde", 2, 2)
```

---

str_len(string : str)

函数说明:

字符串长度。

参数说明:

str1 : 要计算长度的字符串。

返回值:

number，字符串长度。

示例:

```
ret = str_len ("abcde")
```

---

str_find(string : str1, string : str2)

函数说明:

字符串 str1 中首次出现 str2 的位置。

参数说明:

str1 : 要操作的字符串。

str2 : 要操作的字符串。

返回值:

number: 返回位置，从 0 开始计数，未找到返回-1。

示例:

```
ret = str_find ("abcde", "ab")
```

---

str_split(string : str1, string : separator)

函数说明:

按照字符 separator 分割 str1 字符串，返回列表。

参数说明:

str1 : 要操作的字符串。

separator: 分割字符。

返回值:

num_list，字符串列表。

示例:

```
ret = str_split ("abcde","c")
```

---

str_at(string : str1, number : index)

函数说明:

获取字符串 str1 中 index 处的字符，index 从 1 开始。

参数说明:

str1 : 要操作的字符串。

index : 索引，从 1 开始计数。

返回值:

string，index 超出范围返回空字符串。

示例:

```
ret = str_at ("abcde", 2)
```

5.7 辅助函数

list_len (list: value)

函数说明:

该函数返回输入列表的长度。

参数说明:

value: 输入的列表。

返回值:

number: 列表的长度。

示例:

```
ret = list_len ({1,1,1,1,1,1})
```

---

is_valid (value)

函数说明:

该函数判断输入的条件是否合法。

参数说明:

value: 输入的数据，会根据数据类型自动推断，判断逻辑如下

当 value 为 boolean 类型时，true 为合法，false 为非法

当 value 为 list 类型时，列表非空为合法，列表为空为非法

当 value 为 string 类型时，字符非空为合法，字符为空字符串为非法

当 value 为 number 类型时，数字大小非 0 为合法，打字大小为 0 为非法

当 value 为 nil 类型时为非法。

返回值:

false: 非法。

true: 合法。

示例:

```
ret = is_valid ("")
```

---

int16_to_byte_list (number: value)

函数说明:

将 int16 数据类型按照内存结构转换成 byte list。

参数说明:

value: 待转换的 int16 类型数据。

返回值:

转换得到的 byte 数据列表。

示例:

```
ret = int16_to_byte_list (5)
```

---

uint16_to_byte_list (number: value)

函数说明:

将 uint16 数据类型按照内存结构转换成 byte list。

参数说明:

value: 待转换的 uint16 类型数据。

返回值:

转换得到的 byte 数据列表。

示例:

```
ret = uint16_to_byte_list (5)
```

---

int32_to_byte_list (number: value)

函数说明:

将 int32 数据类型按照内存结构转换成 byte list。

参数说明:

value: 待转换的 int32 类型数据。

返回值:

转换得到的 byte 数据列表。

示例:

```
ret = int32_to_byte_list (5)
```

---

uint32_to_byte_list (number: value)

函数说明:

将 uint32 数据类型按照内存结构转换成 byte list。

参数说明:

value: 待转换的 uint32 类型数据。

返回值:

转换得到的 byte 数据列表。

示例:

```
ret = uint32_to_byte_list (5)
```

---

float_to_byte_list (number: value)

函数说明:

将 float 数据类型按照内存结构转换成 byte list。

参数说明:

value: 待转换的 float 类型数据。

返回值:

转换得到的 byte 数据列表。

示例:

```
ret = float_to_byte_list (5.0)
```

---

double_to_byte_list (number: value)

函数说明:

将 double 数据类型按照内存结构转换成 byte list。

参数说明:

value: 待转换的 double 类型数据。

返回值:

转换得到的 byte 数据列表。

示例:

```
ret = int16_to_byte_list (5.0005)
```

---

byte_list_to_int16 (byte_list: value)

函数说明:

将 byte list 数据类型按照内存结构转换成 int16 类型数据。

参数说明:

value: 待转换的 byte list(number list) 类型数据。

返回值:

转换得到的 int16 类型数据。

示例:

```
ret = byte_list_to_int16 ({0x01,0x05})
```

---

byte_list_to_uint16 (byte_list: value)

函数说明:

将 byte list 数据类型按照内存结构转换成 uint16 类型数据。

参数说明:

value: 待转换的 byte list(number list) 类型数据。

返回值:

转换得到的 uint16 类型数据。

示例:

```
ret = byte_list_to_uint16 ({0x01,0x05})
```

---

byte_list_to_int32 (byte_list: value)

函数说明:

将 byte list 数据类型按照内存结构转换成 int32 类型数据。

参数说明:

value: 待转换的 byte list(number list) 类型数据。

返回值:

转换得到的 int32 类型数据。

示例:

```
ret = byte_list_to_int32 ({0x01,0x05})
```

---

byte_list_to_uint32 (byte_list: value)

函数说明:

将 byte list 数据类型按照内存结构转换成 uint32 类型数据。

参数说明:

value: 待转换的 byte list(number list) 类型数据。

返回值:

转换得到的 uint32 类型数据。

示例:

```
ret = byte_list_to_uint32 ({0x01,0x05})
```

---

byte_list_to_float (byte_list: value)

函数说明:

将 byte list 数据类型按照内存结构转换成 float 类型数据。

参数说明:

value: 待转换的 byte list(number list) 类型数据。

返回值:

转换得到的 float 类型数据。

示例:

```
ret = byte_list_to_float ({0x01,0x05})
```

---

byte_list_to_double (byte_list: value)

函数说明:

将 byte list 数据类型按照内存结构转换成 double 类型数据。

参数说明:

value: 待转换的 byte list(number list) 类型数据。

返回值:

转换得到的 double 类型数据。

示例:

```
ret = byte_list_to_double ({0x01,0x05})
```

---

float2word (number: value)

函数说明:

将 float 数据类型按照内存结构转换成长度为 2 的 word 列表类型数据。

参数说明:

value: 待转换的 float 类型数据。

返回值:

转换得到的 word 列表 {word_H, word_L}。

示例:

```
ret = float2word (5.5)
```

---

word2float(number: word_H, number: word_L)

函数说明:

将两个 word 类型数据按照内存结构转换成一个 float 类型数据。

参数说明:

word_H, word_L: 待合并转换的 word 类型数据。

返回值:

转换得到的 float 类型数据。

示例:

```
ret = word2float (5)
```